water quality prediction project

April 10, 2025

[3]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns



# Fixed: Added quotes around the URL
df = pd.read_csv("https://raw.githubusercontent.com/JatinSadhwani02/
↪Water-Quality-Prediction-using-ML-technique/
↪10ee4911a01945c82606a7791760b8fac4c6aa72/water_potability.csv")

# Display the DataFrame
df
```

[3]:
```
            ph    Hardness       Solids  Chloramines     Sulfate  \
0          NaN  204.890456  20791.31898     7.300212  368.516441
1     3.716080  129.422921  18630.05786     6.635246         NaN
2     8.099124  224.236259  19909.54173     9.275884         NaN
3     8.316766  214.373394  22018.41744     8.059332  356.886136
4     9.092223  181.101509  17978.98634     6.546600  310.135738
…        …         …           …           …          …
3271  4.668102  193.681736  47580.99160     7.166639  359.948574
3272  7.808856  193.553212  17329.80216     8.061362         NaN
3273  9.419510  175.762646  33155.57822     7.350233         NaN
3274  5.126763  230.603758  11983.86938     6.303357         NaN
3275  7.874671  195.102299  17404.17706     7.509306         NaN

      Conductivity  Organic_carbon  Trihalomethanes  Turbidity  Potability
0       564.308654       10.379783        86.990970   2.963135           0
1       592.885359       15.180013        56.329076   4.500656           0
2       418.606213       16.868637        66.420093   3.055934           0
3       363.266516       18.436525       100.341674   4.628771           0
4       398.410813       11.558279        31.997993   4.075075           0
…            …             …               …        …          …
3271    526.424171       13.894419        66.687695   4.435821           1
3272    392.449580       19.903225              NaN   2.798243           1
3273    432.044783       11.039070        69.845400   3.298875           1
```

```
3274    402.883113       11.168946        77.488213   4.708658           1
3275    327.459761       16.140368        78.698446   2.309149           1

[3276 rows x 10 columns]
```

[7]:
```
df.shape
```

[7]:
```
(3276, 10)
```

[9]:
```
df.isnull().sum()
```

[9]:
```
ph                 491
Hardness             0
Solids               0
Chloramines          0
Sulfate            781
Conductivity         0
Organic_carbon       0
Trihalomethanes    162
Turbidity            0
Potability           0
dtype: int64
```

[11]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3276 entries, 0 to 3275
Data columns (total 10 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   ph               2785 non-null   float64
 1   Hardness         3276 non-null   float64
 2   Solids           3276 non-null   float64
 3   Chloramines      3276 non-null   float64
 4   Sulfate          2495 non-null   float64
 5   Conductivity     3276 non-null   float64
 6   Organic_carbon   3276 non-null   float64
 7   Trihalomethanes  3114 non-null   float64
 8   Turbidity        3276 non-null   float64
 9   Potability       3276 non-null   int64
dtypes: float64(9), int64(1)
memory usage: 256.1 KB
```

[13]:
```
df.describe()
```

[13]:
```
                ph     Hardness        Solids  Chloramines      Sulfate  \
count  2785.000000  3276.000000   3276.000000  3276.000000  2495.000000
mean      7.080795   196.369496  22014.092526     7.122277   333.775777
```

```
std       1.594320    32.879761   8768.570828     1.583085    41.416840
min       0.000000    47.432000    320.942611     0.352000   129.000000
25%       6.093092   176.850538  15666.690300     6.127421   307.699498
50%       7.036752   196.967627  20927.833605     7.130299   333.073546
75%       8.062066   216.667456  27332.762125     8.114887   359.950170
max      14.000000   323.124000  61227.196010    13.127000   481.030642

       Conductivity  Organic_carbon  Trihalomethanes    Turbidity   Potability
count   3276.000000     3276.000000      3114.000000  3276.000000  3276.000000
mean     426.205111       14.284970        66.396293     3.966786     0.390110
std       80.824064        3.308162        16.175008     0.780382     0.487849
min      181.483754        2.200000         0.738000     1.450000     0.000000
25%      365.734414       12.065801        55.844536     3.439711     0.000000
50%      421.884968       14.218338        66.622485     3.955028     0.000000
75%      481.792305       16.557652        77.337473     4.500320     1.000000
max      753.342620       28.300000       124.000000     6.739000     1.000000
```

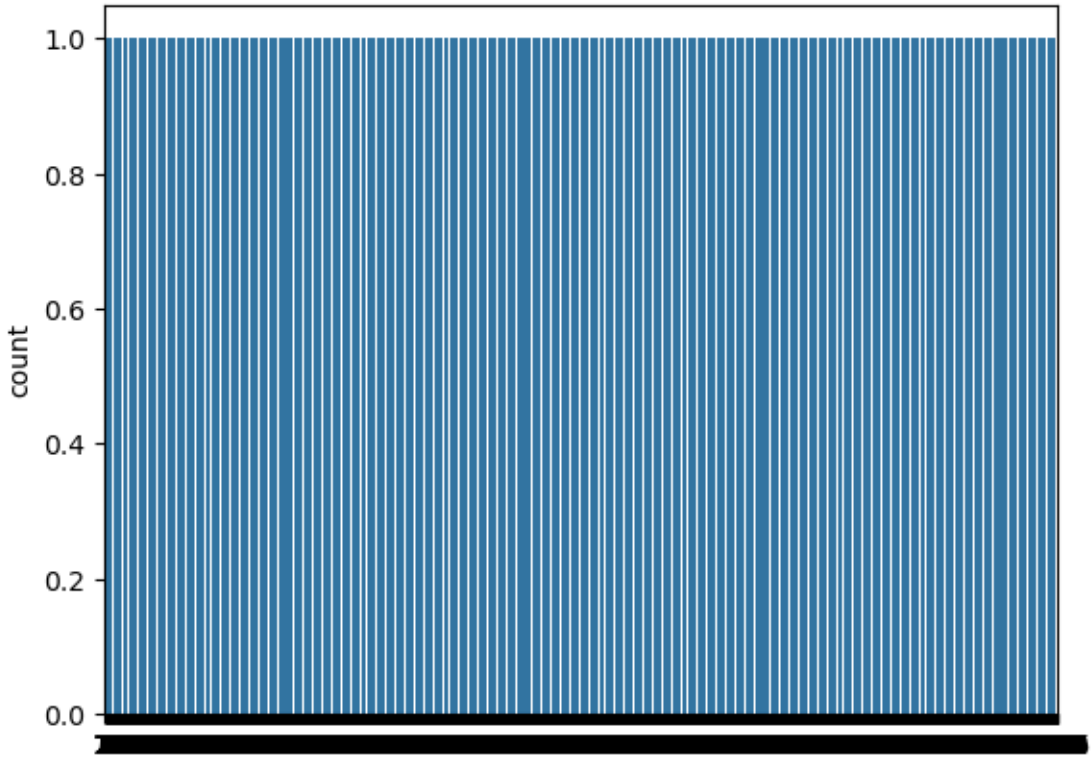
[15]:
```
df.fillna(df.mean(), inplace=True)
df.isnull().sum()
```

[15]:
```
ph                 0
Hardness           0
Solids             0
Chloramines        0
Sulfate            0
Conductivity       0
Organic_carbon     0
Trihalomethanes    0
Turbidity          0
Potability         0
dtype: int64
```

[17]:
```
df.Potability.value_counts()
```

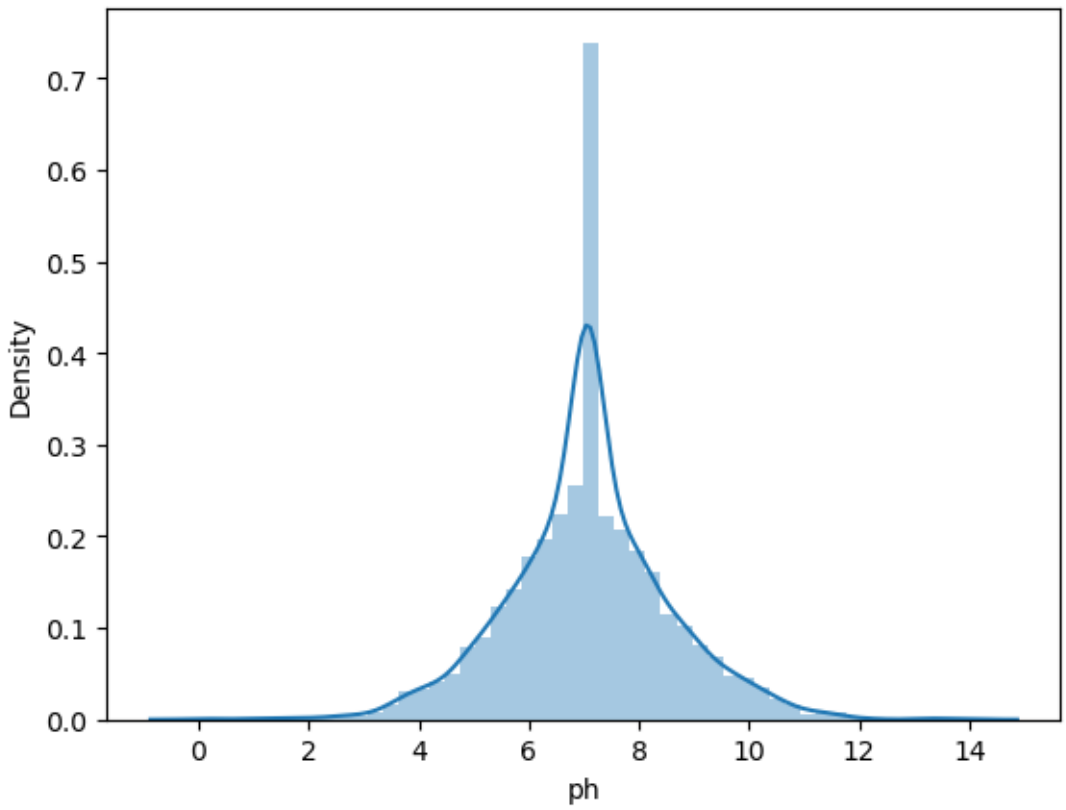
[17]:
```
Potability
0    1998
1    1278
Name: count, dtype: int64
```

[19]:
```
sns.countplot(df['Potability'])
plt.show()
```

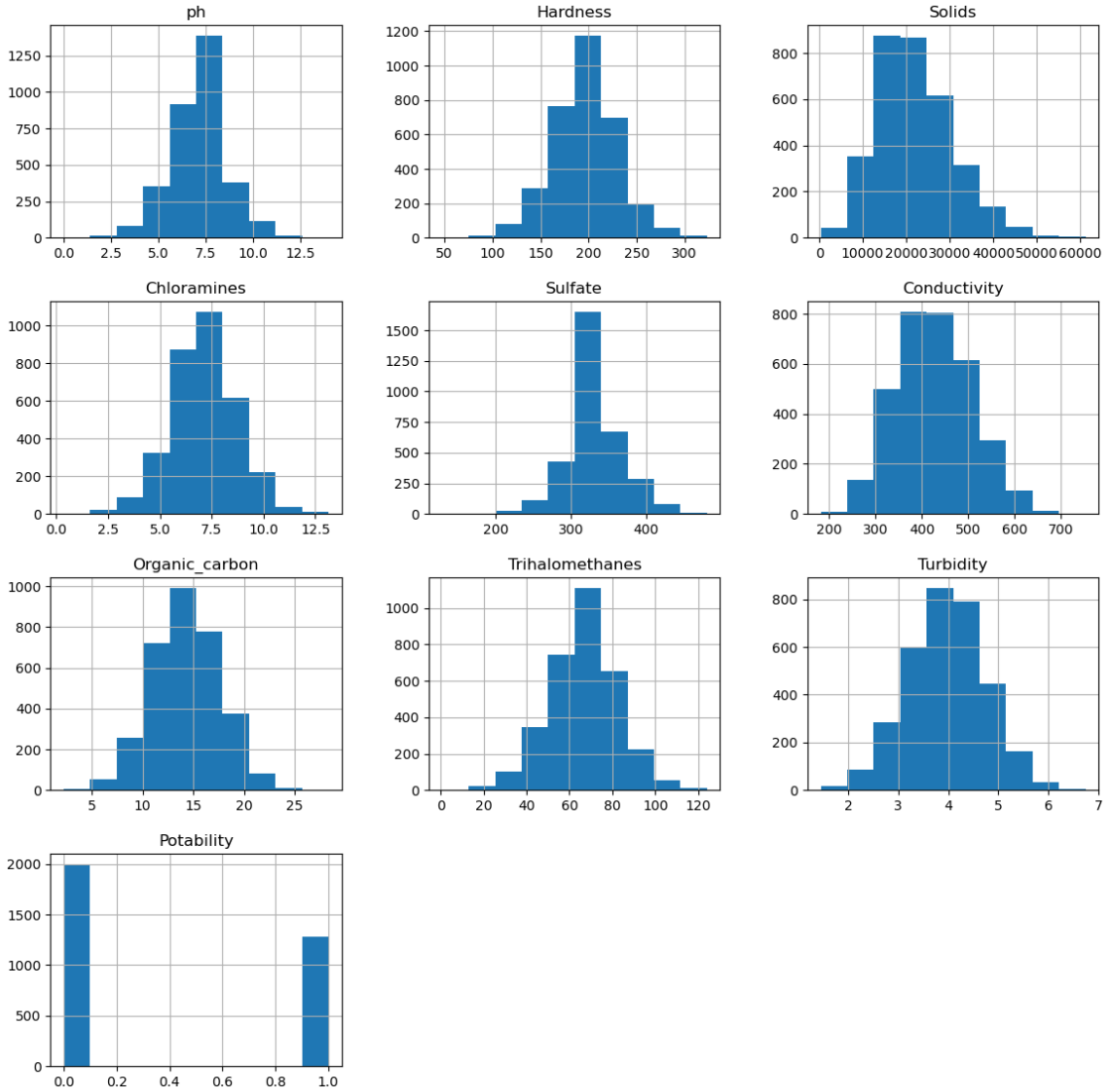


[21]:
```
sns.distplot(df['ph'])
plt.show()
```

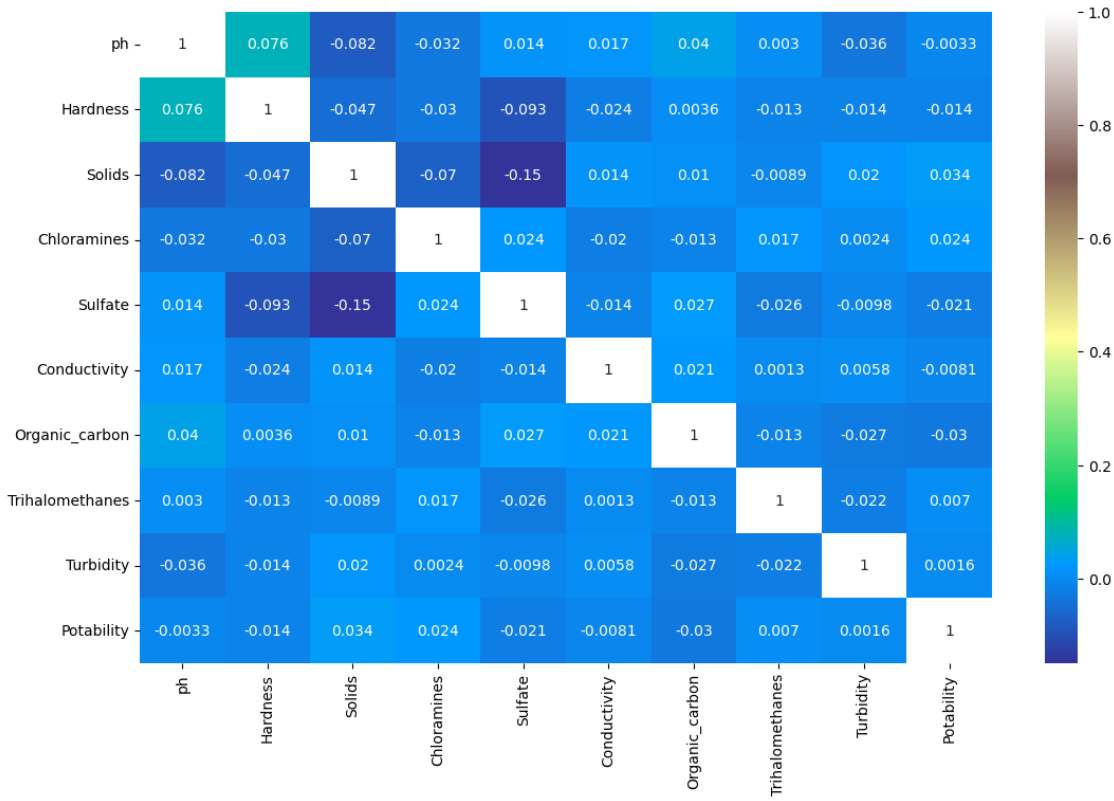
```
C:\Users\KIIT\AppData\Local\Temp\ipykernel_22712\3057384885.py:1: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

  sns.distplot(df['ph'])
```



[23]:
```
df.hist(figsize=(14,14))
plt.show()
```

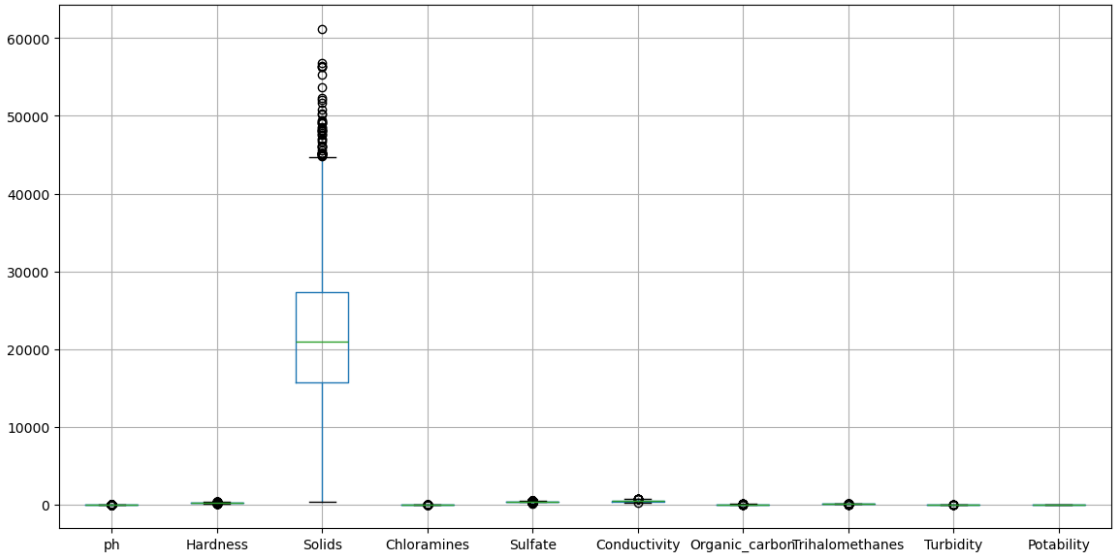


[25]:
```
plt.figure(figsize=(13,8))
sns.heatmap(df.corr(),annot=True,cmap='terrain')
plt.show()
```



[27]:
```
df.boxplot(figsize=(14,7))
```

[27]: <Axes: >



[29]:
```
X = df.drop('Potability',axis=1)
Y= df['Potability']
```

[31]:
```
from sklearn.model_selection import train_test_split
X_train, X_test, Y_train, Y_test = train_test_split(X,Y, test_size= 0.2,␣
↪random_state=101,shuffle=True)
```

[33]:
```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import␣
↪accuracy_score,confusion_matrix,classification_report
dt=DecisionTreeClassifier(criterion= 'gini', min_samples_split= 10, splitter=␣
↪'best')
dt.fit(X_train,Y_train)
```

[33]:
```
DecisionTreeClassifier(min_samples_split=10)
```

[35]:
```
prediction=dt.predict(X_test)
print(f"Accuracy Score = {accuracy_score(Y_test,prediction)*100}")
print(f"Confusion Matrix =\n {confusion_matrix(Y_test,prediction)}")
print(f"Classification Report =\n {classification_report(Y_test,prediction)}")
```

```
Accuracy Score = 58.993902439024396
Confusion Matrix =
 [[276 126]
 [143 111]]
Classification Report =
               precision    recall  f1-score   support

           0       0.66      0.69      0.67       402
           1       0.47      0.44      0.45       254

    accuracy                           0.59       656
   macro avg       0.56      0.56      0.56       656
weighted avg       0.59      0.59      0.59       656
```

[41]:
```
res = dt.predict([[5.735724, 158.318741,25363.016594,7.728601,377.543291,568.
↪304671,13.626624,75.952337,4.732954]])[0]
res
```

```
C:\Users\KIIT\anaconda3\Lib\site-packages\sklearn\base.py:493: UserWarning: X
does not have valid feature names, but DecisionTreeClassifier was fitted with
feature names
  warnings.warn(
```

[41]:
```
1
```

[43]:
```
from sklearn.model_selection import RepeatedStratifiedKFold
from sklearn.model_selection import GridSearchCV

# define models and parameters
model = DecisionTreeClassifier()
criterion = ["gini", "entropy"]
splitter = ["best", "random"]
min_samples_split = [2,4,6,8,10,12,14]

# define grid search
grid = dict(splitter=splitter, criterion=criterion,␣
 ↪min_samples_split=min_samples_split)
cv = RepeatedStratifiedKFold(n_splits=10, n_repeats=3, random_state=1)
grid_search_dt = GridSearchCV(estimator=model, param_grid=grid, n_jobs=-1,␣
 ↪cv=cv,
                            scoring='accuracy',error_score=0)
grid_search_dt.fit(X_train, Y_train)
```

[43]:
```
GridSearchCV(cv=RepeatedStratifiedKFold(n_repeats=3, n_splits=10,
random_state=1),
             error_score=0, estimator=DecisionTreeClassifier(), n_jobs=-1,
             param_grid={'criterion': ['gini', 'entropy'],
                         'min_samples_split': [2, 4, 6, 8, 10, 12, 14],
                         'splitter': ['best', 'random']},
             scoring='accuracy')
```

[45]:
```
print(f"Best: {grid_search_dt.best_score_:.3f} using {grid_search_dt.
 ↪best_params_}")
means = grid_search_dt.cv_results_['mean_test_score']
stds = grid_search_dt.cv_results_['std_test_score']
params = grid_search_dt.cv_results_['params']

for mean, stdev, param in zip(means, stds, params):
    print(f"{mean:.3f} ({stdev:.3f}) with: {param}")

print("Training Score:",grid_search_dt.score(X_train, Y_train)*100)
print("Testing Score:", grid_search_dt.score(X_test, Y_test)*100)
```

```
Best: 0.596 using {'criterion': 'entropy', 'min_samples_split': 14, 'splitter':
'random'}
0.580 (0.032) with: {'criterion': 'gini', 'min_samples_split': 2, 'splitter':
'best'}
0.583 (0.034) with: {'criterion': 'gini', 'min_samples_split': 2, 'splitter':
'random'}
0.586 (0.028) with: {'criterion': 'gini', 'min_samples_split': 4, 'splitter':
'best'}
0.574 (0.031) with: {'criterion': 'gini', 'min_samples_split': 4, 'splitter':
'random'}
```

```
0.584 (0.032) with: {'criterion': 'gini', 'min_samples_split': 6, 'splitter':
'best'}
0.580 (0.034) with: {'criterion': 'gini', 'min_samples_split': 6, 'splitter':
'random'}
0.587 (0.029) with: {'criterion': 'gini', 'min_samples_split': 8, 'splitter':
'best'}
0.585 (0.035) with: {'criterion': 'gini', 'min_samples_split': 8, 'splitter':
'random'}
0.590 (0.026) with: {'criterion': 'gini', 'min_samples_split': 10, 'splitter':
'best'}
0.591 (0.024) with: {'criterion': 'gini', 'min_samples_split': 10, 'splitter':
'random'}
0.588 (0.028) with: {'criterion': 'gini', 'min_samples_split': 12, 'splitter':
'best'}
0.591 (0.029) with: {'criterion': 'gini', 'min_samples_split': 12, 'splitter':
'random'}
0.589 (0.026) with: {'criterion': 'gini', 'min_samples_split': 14, 'splitter':
'best'}
0.592 (0.030) with: {'criterion': 'gini', 'min_samples_split': 14, 'splitter':
'random'}
0.588 (0.030) with: {'criterion': 'entropy', 'min_samples_split': 2, 'splitter':
'best'}
0.569 (0.026) with: {'criterion': 'entropy', 'min_samples_split': 2, 'splitter':
'random'}
0.584 (0.028) with: {'criterion': 'entropy', 'min_samples_split': 4, 'splitter':
'best'}
0.586 (0.027) with: {'criterion': 'entropy', 'min_samples_split': 4, 'splitter':
'random'}
0.584 (0.030) with: {'criterion': 'entropy', 'min_samples_split': 6, 'splitter':
'best'}
0.583 (0.037) with: {'criterion': 'entropy', 'min_samples_split': 6, 'splitter':
'random'}
0.585 (0.027) with: {'criterion': 'entropy', 'min_samples_split': 8, 'splitter':
'best'}
0.585 (0.028) with: {'criterion': 'entropy', 'min_samples_split': 8, 'splitter':
'random'}
0.587 (0.030) with: {'criterion': 'entropy', 'min_samples_split': 10,
'splitter': 'best'}
0.590 (0.035) with: {'criterion': 'entropy', 'min_samples_split': 10,
'splitter': 'random'}
0.587 (0.029) with: {'criterion': 'entropy', 'min_samples_split': 12,
'splitter': 'best'}
0.593 (0.027) with: {'criterion': 'entropy', 'min_samples_split': 12,
'splitter': 'random'}
0.588 (0.029) with: {'criterion': 'entropy', 'min_samples_split': 14,
'splitter': 'best'}
0.596 (0.032) with: {'criterion': 'entropy', 'min_samples_split': 14,
'splitter': 'random'}
```

```
Training Score: 82.06106870229007
Testing Score: 60.36585365853659
```

[ ]:

```

```